Phase 6 Completion Report: SQL Data Quality Validation

1. Phase Overview

Phase 6 of the Banking Data Governance & Data Quality Pipeline focused on SQL-based data quality validation and reconciliation against the curated warehouse layer.

The goal of this phase was to practice practical SQL data quality work that is common in data governance, data quality engineering, financial reporting, and warehouse validation environments. The project already had PostgreSQL source data, raw Parquet files, standardized Parquet outputs, and curated warehouse-style fact and dimension models. Phase 6 added the SQL validation layer on top of those curated outputs.

The Phase 6 validation flow is:

PostgreSQL banking source system
→ Python ingestion layer
→ raw Parquet zone
→ PySpark standardized processing layer
→ standardized Parquet zone
→ curated warehouse layer
→ DuckDB SQL validation layer
→ SQL data quality results and logs

Phase 6 intentionally stayed focused on SQL data quality validation. It did not include PySpark data quality rule execution, OpenMetadata setup, governance YAML artifacts, lineage YAML files, Agile/JIRA files, business glossary work, or stewardship automation. Those items remain assigned to later phases.

2. Phase 6 Objective

The objective of Phase 6 was to build a SQL-based validation and reconciliation layer that checks the curated warehouse outputs for data quality issues.

The main validation categories were:

- Null checks
- Duplicate logical key checks
- Accepted-value checks
- Referential-integrity checks
- Source-to-target reconciliation
- Control-total validation
- Exception-record queries

The main validation target was the curated warehouse layer:

1. dim_customer
2. dim_account
3. dim_branch
4. dim_product
5. fact_transaction
6. fact_customer_risk_score
7. fact_daily_account_balance

The standardized/source-like comparison tables were also used where needed:

1. customers
2. accounts
3. transactions
4. branches
5. products
6. customer_risk_scores
7. reference_codes

3. SQL Execution Design Decision

The project currently stores curated and standardized outputs as Parquet files rather than database tables. Because of this, Phase 6 needed a practical SQL execution approach.

The selected solution was DuckDB.

DuckDB was chosen because it can query Parquet files directly using SQL. This allowed the project to practice SQL validation without adding unnecessary database-loading complexity.

The evaluated options were:

DuckDB querying Parquet directly:
Recommended because it is simple, local, SQL-first, and works directly with Parquet files.

Spark SQL through PySpark:
Not selected because Phase 6 was meant to stay SQL-focused. PySpark data quality validation belongs to Phase 7.

Loading Parquet into PostgreSQL:
Not selected because it would require extra table creation, type mapping, loading logic, and cleanup work. That would distract from the SQL data quality objective.

Other local SQL options:
Not selected because DuckDB has stronger direct Parquet support.

Final design decision:
Use an in-memory DuckDB connection and create temporary SQL views over curated and standardized Parquet outputs.

DuckDB was used only as the SQL execution engine. It was not introduced as a new persistent warehouse layer.

4. Phase 6 Repository Additions

Phase 6 added the SQL validation structure and supporting Python runner.

Key repository additions included:

- DuckDB dependency in requirements.txt
- SQL validation runner:
    - python/validation/run_sql_validation.py
- SQL validation scripts:
    - sql/validation/01_null_checks.sql
    - sql/validation/02_duplicate_checks.sql
    - sql/validation/03_accepted_values_checks.sql
    - sql/validation/04_referential_integrity_checks.sql
    - sql/validation/05_source_target_reconciliation.sql
    - sql/validation/06_control_totals.sql
    - sql/validation/07_exception_records.sql
- SQL validation documentation:
    - docs/sql_data_quality_validation.md
- README updates with Phase 6 commands
- Runtime output and log ignore rules for:
    - data/dq_results/
    - logs/sql/

Runtime output locations:

- data/dq_results/sql/load_date=2026-06-16/
- logs/sql/

These runtime folders are generated locally and are ignored by Git.

5. Phase 6A: DuckDB SQL Validation Foundation

Phase 6A Goal

The goal of Phase 6A was to create the SQL validation foundation using DuckDB.

Phase 6A focused on the core quality checks:

- Null checks
- Duplicate checks
- Accepted-value checks
- Referential-integrity checks

Phase 6A Work Completed

Phase 6A added DuckDB as the SQL execution engine and created a Python runner to execute SQL validation scripts.

The runner supports:

- In-memory DuckDB execution
- Load-date validation
- Curated Parquet view creation
- Standardized Parquet view creation
- SQL file execution
- Optional selected-check execution
- Result CSV generation
- Summary CSV generation
- Console and file logging
- Non-zero failure behavior only for runner/configuration/execution errors

The runner creates DuckDB views over the curated models:

- dim_customer
- dim_account
- dim_branch
- dim_product
- fact_transaction

- fact_customer_risk_score
- fact_daily_account_balance

It also creates views over standardized tables using a std_ prefix:

- std_customers
- std_accounts
- std_transactions
- std_branches
- std_products
- std_customer_risk_scores
- std_reference_codes

Phase 6A SQL Scripts

The following SQL scripts were created in Phase 6A:

1. 01_null_checks.sql
   Validates that critical fields and required keys are not null.
2. 02_duplicate_checks.sql
   Validates that logical primary keys and composite keys do not contain duplicates.
3. 03_accepted_values_checks.sql
   Validates that key status, type, category, and risk fields contain expected values.
4. 04_referential_integrity_checks.sql
   Validates that fact and dimension relationships do not contain orphan keys.

Each SQL script returns a standard result structure:

- check_category
- check_name
- table_name
- column_name
- severity
- failed_record_count
- status
- details

Phase 6A Validation Results

Phase 6A validation completed successfully.

Results:

01_null_checks.sql: 8 checks, 0 failed
02_duplicate_checks.sql: 7 checks, 0 failed
03_accepted_values_checks.sql: 9 checks, 0 failed
04_referential_integrity_checks.sql: 14 checks, 0 failed

Generated outputs:

data/dq_results/sql/load_date=2026-06-16/
logs/sql/sql_validation_summary_20260618_153302.csv

Phase 6A commit:

371ca33 Phase 6A - Add DuckDB SQL validation foundation

Final Git status after Phase 6A was clean.

6. Phase 6B: Reconciliation, Control Totals, and Exception Records

Phase 6B Goal

The goal of Phase 6B was to extend the SQL validation layer with audit-style reconciliation and exception reporting.

Phase 6B focused on:

- Source-to-target reconciliation
- Control-total validation
- Exception-record queries
- SQL validation documentation
- README command updates
- Runner support for exception-record result schemas

Phase 6B Work Completed

Phase 6B added three additional SQL validation files:

1. 05_source_target_reconciliation.sql
   Compares standardized/source-like tables to curated warehouse outputs.
2. 06_control_totals.sql
   Validates financial and count-based totals across standardized and curated layers.
3. 07_exception_records.sql
   Returns actual exception rows instead of only summary counts.

Phase 6B also added documentation explaining the purpose of the SQL validation layer and how each validation category supports governance and data quality practice.

Source-to-Target Reconciliation

The reconciliation checks compare source-like standardized data to curated outputs.

Examples of reconciliation coverage:

- std_customers row count compared to dim_customer
- std_accounts row count compared to dim_account
- std_branches row count compared to dim_branch
- std_products row count compared to dim_product
- std_transactions row count compared to fact_transaction
- std_customer_risk_scores row count compared to fact_customer_risk_score
- transaction_id reconciliation between source transactions and fact_transaction
- account_id reconciliation between source accounts and dim_account
- customer_id reconciliation between source customers and dim_customer

These checks confirm that the curated layer did not unexpectedly lose records during transformation.

Control Totals

Control-total checks validate totals and grouped counts between standardized and curated outputs.

Examples of control-total coverage:

- Total transaction amount in standardized transactions compared to fact_transaction
- Transaction counts by available transaction fields
- Account counts by available account fields
- Customer counts by branch where available
- Balance-related comparisons where available

These checks simulate common financial data validation patterns where row counts alone are not enough. In banking and financial reporting, amount-level control totals are important because a dataset can have the correct row count but still have incorrect values.

Exception Records

Exception-record queries return actual failed records rather than only summary counts.

Examples of exception coverage:

- Orphan transaction account IDs

- Orphan transaction customer IDs
- Orphan transaction product IDs
- Duplicate transaction IDs
- Null critical keys
- Invalid transaction amount records
- Source transactions missing from fact_transaction
- Source accounts missing from dim_account
- Source customers missing from dim_customer

Each exception query is bounded to avoid producing large result sets. This simulates a realistic data quality investigation process where analysts and data stewards review a sample of failed records.

Runner Update

The SQL validation runner was updated to support exception-record result schemas. This was needed because summary validation checks return count/status-style rows, while exception-record queries return actual data-quality issue rows.

This made the runner more flexible while still keeping Phase 6 focused on SQL validation.

7. Phase 6B Validation Results

Phase 6B validation completed successfully.

Results:

Reconciliation: 9 checks, 0 failed
Control totals: 7 checks, 0 failed
Exception records: 0 rows
Full suite: 54 checks, 0 failed

Generated outputs:

data/dq_results/sql/load_date=2026-06-16/
logs/sql/sql_validation_summary_20260618_154517.csv

Phase 6B commit:

1830481 Phase 6B - Add SQL reconciliation and exception checks

Final Git status after Phase 6B was clean.

No Phase 6 tag was created at the time of commit. The recommended final tag after approval is:

phase-6-complete

8. Final Phase 6 Status

Phase 6 is complete.

Final status:

Phase 6A DuckDB SQL validation foundation: Complete
Phase 6B reconciliation and exception checks: Complete
DuckDB dependency added: Complete
SQL runner created: Complete
Curated Parquet views created: Complete
Standardized Parquet views created: Complete
Null checks: Complete
Duplicate checks: Complete
Accepted-value checks: Complete
Referential-integrity checks: Complete
Source-to-target reconciliation: Complete
Control totals: Complete
Exception-record queries: Complete
SQL result CSV outputs: Complete
SQL summary logs: Complete
SQL validation documentation: Complete
README updates: Complete
Runtime folders ignored by Git: Complete
Phase 6A commit completed: Complete
Phase 6B commit completed: Complete
Final Git status: Clean
Phase 6 tag: Pending if not already created

Recommended tag command:

git tag phase-6-complete

9. Validation Commands Used

The runner was compiled using:

python -m py_compile python\validation\run_sql_validation.py

Phase 6A selected check validation:

python python\validation\run_sql_validation.py --load-date 2026-06-16 --checks 01_null_checks.sql

Phase 6B selected check validation:

python python\validation\run_sql_validation.py --load-date 2026-06-16 --checks
05_source_target_reconciliation.sql 06_control_totals.sql 07_exception_records.sql

Full SQL suite validation:

python python\validation\run_sql_validation.py --load-date 2026-06-16

PowerShell inspection commands:

Get-ChildItem .\data\dq_results\sql\load_date=2026-06-16

$latestSummary = Get-ChildItem .\logs\sql\sql_validation_summary_*.csv | Sort-Object LastWriteTime -Descending | Select-Object -First 1
Import-Csv $latestSummary.FullName | Format-Table

Git status validation:

git status --short

10. Git History After Phase 6

Phase 6A commit:

371ca33 Phase 6A - Add DuckDB SQL validation foundation

Phase 6B commit:

1830481 Phase 6B - Add SQL reconciliation and exception checks

Expected final tagging state after closeout:

1830481 (HEAD -> main, tag: phase-6-complete) Phase 6B - Add SQL reconciliation and exception checks
371ca33 Phase 6A - Add DuckDB SQL validation foundation
3fcb13d (tag: phase-5-complete) Phase 5B - Build curated warehouse models

11. Skills Practiced in Phase 6

Phase 6 helped practice the following skills:

- SQL-based data quality validation
- DuckDB SQL execution over Parquet
- Data warehouse validation
- Null checking
- Duplicate key detection
- Accepted-value validation
- Referential-integrity validation
- Source-to-target reconciliation
- Control-total validation
- Financial amount reconciliation
- Exception-record generation

- Data quality result reporting
- SQL automation using Python
- Runtime logging
- Summary CSV reporting
- Validation evidence generation
- Data governance support through measurable quality checks
- SDLC-style phased delivery
- Git commit discipline

12. Governance and Data Quality Relevance

Phase 6 is important because data governance is not only about cataloging data assets. Governance also requires measurable controls over data quality.

This phase connects technical validation to governance practice in the following ways:

- Critical fields were checked for nulls.
- Logical keys were checked for duplicates.
- Controlled fields were checked for valid values.
- Fact-to-dimension relationships were checked for orphan records.
- Source-to-target row movement was reconciled.
- Financial totals were validated using control totals.
- Exception-record queries were created for investigation.
- Results were written to repeatable output files.
- Logs and summary CSVs provided validation evidence.

This is the practical SQL work behind enterprise data quality programs. In a real organization, similar checks would support audit readiness, data steward review, issue triage, regulatory reporting, and trust in curated warehouse datasets.

13. Interview Explanation

A clear interview explanation for Phase 6 would be:

“In Phase 6, I built the SQL-based data quality validation layer for my banking governance pipeline. Since the curated warehouse outputs were stored as Parquet files, I used DuckDB to query the Parquet files directly with SQL instead of loading them into another database. I created a Python runner that builds temporary DuckDB views over the curated and standardized layers, executes SQL validation scripts, writes result CSVs, and generates a summary log.

The SQL suite includes null checks, duplicate logical key checks, accepted-value checks, referential-integrity checks, source-to-target reconciliation, control totals, and exception-record queries. The full suite produced 54 checks with 0 failures, and the exception-record query returned 0 rows. This gave me hands-on practice with SQL data quality validation, reconciliation, control totals, and audit-style exception reporting in a warehouse environment."

14. Phase 6 Conclusion

Phase 6 successfully added a SQL data quality validation layer to the Banking Data Governance & Data Quality Pipeline.

The project now has a complete flow from source data generation through SQL validation:

PostgreSQL banking source system
→ Python ingestion layer
→ raw Parquet zone
→ PySpark standardized processing layer
→ standardized Parquet zone
→ curated warehouse fact and dimension layer
→ DuckDB SQL data quality validation layer

Phase 6 confirmed that the curated warehouse outputs passed all SQL validation categories implemented in this phase:

- 54 total checks
- 0 failed checks
- 0 exception records

The project is now ready for the next phase:

Phase 7: PySpark Data Quality Validation

Phase 7 should focus on PySpark-based data profiling and validation. It should not repeat the SQL validation work from Phase 6. Instead, it should practice scalable PySpark data quality checks, profiling outputs, and rule execution against the same banking datasets.